Survival stuff

Henrik Renlund

April 30, 2026

Contents

1 Toy data and identification of time-to-event variables

First we generate some data to apply our functions on:

```
n <- 1000
rnd <- function(set) sample(set, size = n, replace = TRUE)
set.seed(20260424)
d <- data.table(
    id = 1:n,
    age = runif(n, 40, 80),
    sex = rnd(c("M", "F")),
    group = factor(rnd(LETTERS[1:2]))
)
d[, `:=`(foo = rexp(n, 1/(1900 + 500 * as.integer(sex == "M") -
                          300 * as.integer(group) +
                          2* age)),
         bar = rexp(n, 1/(2300 + 700 * as.integer(sex == "M") -
                          250 * as.integer(group) +
                          2.5 * age)),
         cens = runif(n, min = 180, max = 810)
)]
d[, `:=`(ev.Foo = fifelse(foo <= cens, 1L, 0L),
         t.Foo = round(fifelse(foo <= cens, foo, cens)),
         ev.Bar = fifelse(bar <= cens, 1L, 0L),
         t.Bar = round(fifelse(bar <= cens, bar, cens)))]
d[, c("foo", "bar", "cens") := NULL]
str(d)

## Classes 'data.table' and 'data.frame': 1000 obs. of  8 variables:
##  $ id    : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ age   : num  42.4 56.5 57.8 50.6 52.8 ...
##  $ sex   : chr  "M" "M" "M" "F" ...
##  $ group : Factor w/ 2 levels "A","B": 2 1 1 2 1 2 2 2 1 1 ...
##  $ ev.Foo: int  0 0 1 0 0 1 0 1 0 1 ...
##  $ t.Foo : num  770 365 149 216 694 277 389 114 404 244 ...
##  $ ev.Bar: int  0 0 0 0 0 0 0 1 0 0 ...
##  $ t.Bar : num  770 365 366 216 694 491 389 392 404 696 ...
##  - attr(*, ".internal.selfref")=<pointer: 0x0000023269838dc0>
```

`Important note:` some functions can automatically detect time-to-event variable pairs if they have a common pre- or suffix. By default this is `t.` for the time- and `ev.` for the status component. This can be changed within the options (documentation needed?).

If you do not have this structure, you will need to give an "survival table" to the the `surv` argument of these functions. This is a data frame with columns `time`, `event`, and `label`. For the test data set below this would be

```
extract_stab_from_names(names(d))

##   label  time  event
## 1   Foo t.Foo ev.Foo
## 2   Bar t.Bar ev.Bar
```

2 Wrappers for survfit

The function `survival::survfit` returns a list that is hard to work with directly. The `suirvfitted` function is a simple wrapper that tidies the output

```
survfitted(Surv(t.Foo, ev.Foo) ~ group, data = d) |> str()

## Classes 'data.table' and 'data.frame': 732 obs. of  12 variables:
##  $ time     : num  0 1 9 11 15 26 32 37 44 49 ...
##  $ estimate : num  1 0.996 0.994 0.99 0.988 ...
##  $ surv     : num  1 0.996 0.994 0.99 0.988 ...
##  $ ci.low   : num  1 0.99 0.987 0.981 0.978 ...
##  $ ci.high  : num  1 1 1 0.999 0.998 ...
##  $ n.risk   : num  494 494 492 491 489 488 487 485 484 482 ...
##  $ n.event  : num  0 2 1 2 1 1 2 1 2 1 ...
##  $ n.censor : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ std.error: num  0 0.00287 0.00352 0.00455 0.00499 ...
##  $ cumhaz   : num  0 0.00405 0.00608 0.01015 0.0122 ...
##  $ std.chaz : num  0 0.00286 0.00351 0.00454 0.00498 ...
##  $ group    : Factor w/ 2 levels "A","B": 1 1 1 1 1 1 1 1 1 1 ...
##  - attr(*, ".internal.selfref")=<pointer: 0x0000023269838dc0>
```

The function `survfit_galore` is an extension of this idea that can handle multiple outcomes and groupings at once. Groups defined by variables are given in the formula argument, whereas a grouping that can contain overlaps has to be given in the "grouping table" (`gtab`) argument

```
gt <- d[, .("A / B" = group %in% LETTERS[1:2],
            "B / C" = group %in% LETTERS[2:3])]
sg <- survfit_galore( ~ sex, data = d, gtab = gt)
```

Note that we did not specify a left hand side of the formula. As all time-to-event components are identifiable with the default system of prefixes, all time-to-event variables are used by default.

The output we get here is easy to use immediately for e.g. plotting

```
xyplot(estimate ~ time | group + outcome, data = sg, group = sex,
       type = c("s", "g"), lwd = 2, xlim = c(-10,740),
       auto.key = list(space = "top"),
       xlab = "Days since index",
       ylab = "Kaplain-Meier estimate") |>
    latticeExtra::useOuterStrips()
```

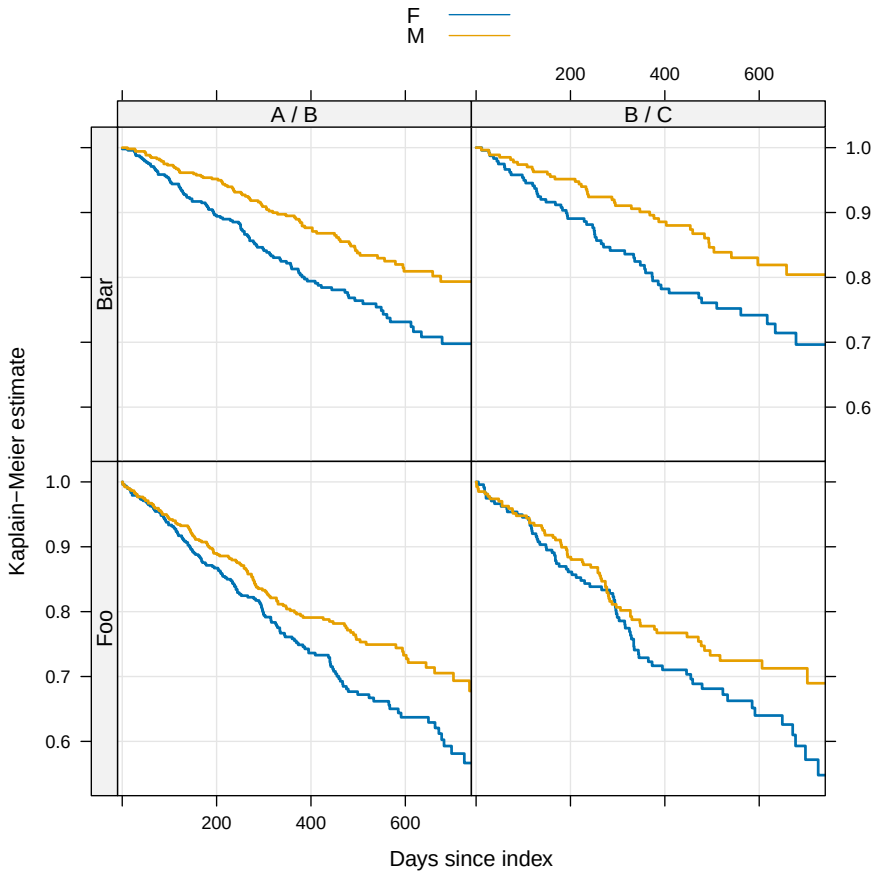

Figure 1: An easy plot to make!

3 Time-to-event variable transformations

A reference plot for the time-to-event variables in the toy data set

```
xyplot(1-estimate ~ time, data = survfit_galore( ~ 1, data = d),
       group = outcome, auto.key = list(space = "top"), type = c("s"), lwd = 2,
       xlab = "Days since index", ylab = "Inverted Kaplan-Meier estimate",
       xlim = c(-10, 740), ylim = c(-0.05,1.05))
```

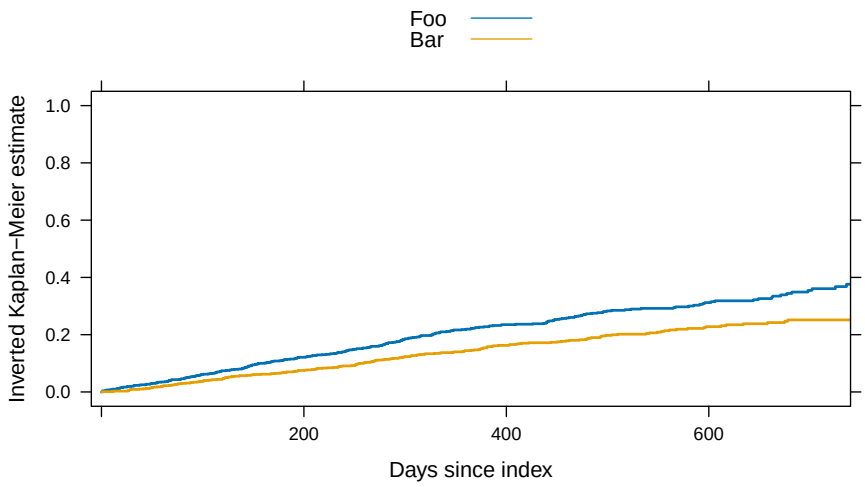

Figure 2: Reference plot for Foo and Bar.

3.1 Change the time scale

A common transformation is to change the time scale of the time component of the time-to event data. Here we have the test data time units in days, and we want years. As all time-to-event components are identifiable with the default system of prefixes, this is easily achieved

```
D <- rescale_surv(data = d, FUN = \(x) x/365.25, strip = FALSE)
str(D)

## Classes 'data.table' and 'data.frame': 1000 obs. of  8 variables:
##  $ id    : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ age   : num  42.4 56.5 57.8 50.6 52.8 ...
##  $ sex   : chr  "M" "M" "M" "F" ...
##  $ group : Factor w/ 2 levels "A","B": 2 1 1 2 1 2 2 2 1 1 ...
##  $ ev.Foo: int  0 0 1 0 0 1 0 1 0 1 ...
##  $ t.Foo : num  2.108 0.999 0.408 0.591 1.9 ...
##  $ ev.Bar: int  0 0 0 0 0 0 0 1 0 0 ...
##  $ t.Bar : num  2.108 0.999 1.002 0.591 1.9 ...
##  - attr(*, ".internal.selfref")=<pointer: 0x0000023269838dc0>
```

Note that `strip = FALSE` lets ut keep the entire data set (with only the time components updated).

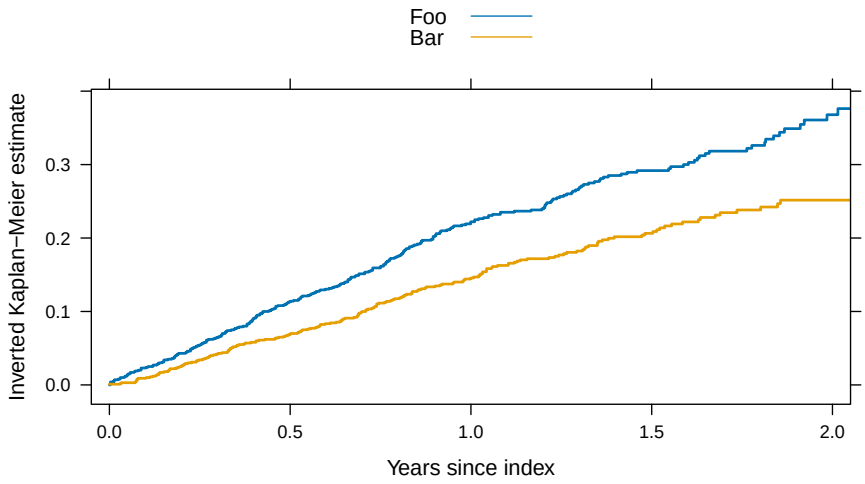

Figure 3: Changing the time scale.

3.2 Combined outcomes

Sometimes one wants to combine two time-to-event variables into a single one.

```
cs <- combine_surv(surv = c("Foo", "Bar"), data = D, strip = FALSE)
str(cs)

## Classes 'data.table' and 'data.frame': 1000 obs. of  10 variables:
##  $ id          : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ age         : num  42.4 56.5 57.8 50.6 52.8 ...
##  $ sex         : chr  "M" "M" "M" "F" ...
##  $ group       : Factor w/ 2 levels "A","B": 2 1 1 2 1 2 2 2 1 1 ...
##  $ ev.Foo      : int  0 0 1 0 0 1 0 1 0 1 ...
##  $ t.Foo       : num  2.108 0.999 0.408 0.591 1.9 ...
##  $ ev.Bar      : int  0 0 0 0 0 0 0 1 0 0 ...
##  $ t.Bar       : num  2.108 0.999 1.002 0.591 1.9 ...
##  $ t.Combined : num  2.108 0.999 0.408 0.591 1.9 ...
##  $ ev.Combined: int  0 0 1 0 0 1 0 1 0 1 ...
##  - attr(*, ".internal.selfref")=<pointer: 0x0000023269838dc0>
```

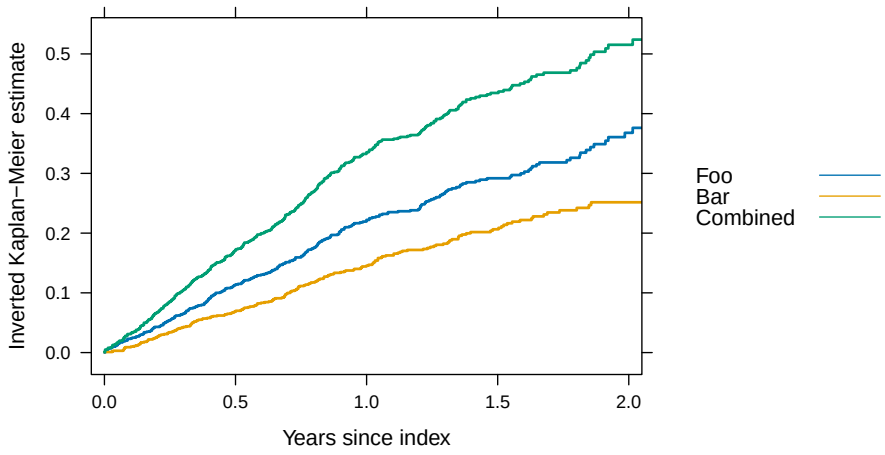

Figure 4: Combining time-to-event data into a single variable.

3.3 Truncation and landmarking

There are functions for truncating (`truncate_surv`) and landmarking (`landmark_surv`) time-to-event variables. In the example below we do not reset the landmark data to 0 (as is the default) to illustrate what is going on.

```
ts <- truncate_surv(data = D, trunc = 1, strip = FALSE)
ls <- landmark_surv(data = D, landmark = 1,
                    strip = FALSE, reset = FALSE)
R <- rbind(ts[, type := "truncation"], ls[, type := "landmark"])
r <- survfit_galore( ~ type, data = R, t0 = FALSE)
```

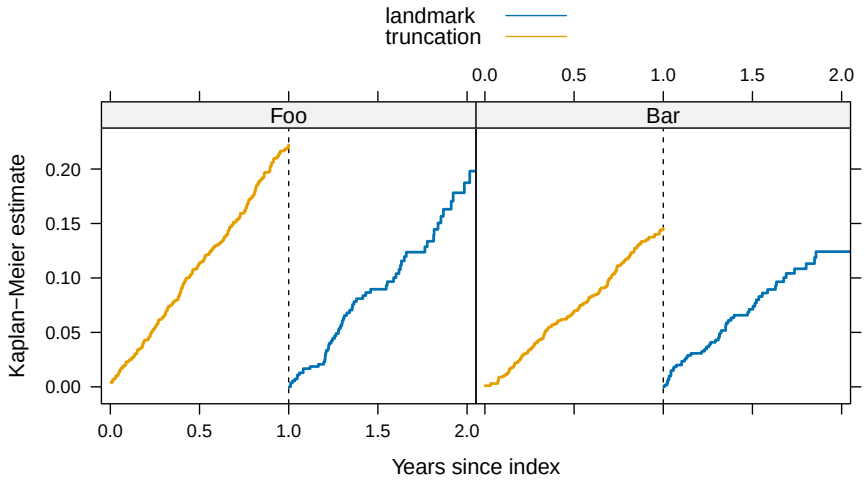

Figure 5: Effect of truncating and landmarking.

3.4 Surv:ifying time-to-event variables

There is a function to convert time-to-event pairs into objects of class `Surv`.

```
str(Survclass_surv(data = D, strip = FALSE))
```

```
## 'data.frame': 1000 obs. of  6 variables:
##  $ id   : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ age  : num  42.4 56.5 57.8 50.6 52.8 ...
##  $ sex  : chr  "M" "M" "M" "F" ...
##  $ group: Factor w/ 2 levels "A","B": 2 1 1 2 1 2 2 2 1 1 ...
##  $ Foo  : 'Surv' num [1:1000, 1:2] 2.10815+ 0.99932+ 0.40794  0.59138+ 1.90007+ 0.75838
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : NULL
##   .. ..$ : chr [1:2] "time" "status"
##   ..- attr(*, "type")= chr "right"
##  $ Bar  : 'Surv' num [1:1000, 1:2] 2.1081+ 0.9993+ 1.0021+ 0.5914+ 1.9001+ 1.3443+ 1.065
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : NULL
##   .. ..$ : chr [1:2] "time" "status"
##   ..- attr(*, "type")= chr "right"
```

4 Related to Cox regression

4.1 Toy observational data

Table 1: Toy obesrvational data

	ctrl	**case**
	n = 624	n = 376
Covariates		
male: Female	291 (46.6%)	205 (54.5%)
Male	333 (53.4%)	171 (45.5%)
age	62.3 (52.8 - 71.7)	53.6 (47.1 - 63.1)
old: 1	374 (59.9%)	126 (33.5%)
Time-to-event		
U	195; 0.071	119; 0.071

Rows: 1000 (ctrl:624, case:376). No duplicate subjects. Categorical variable: Count (Percent). [n] is missing count (if applicable). Numeric variable: Median (Q1-Q3). Time-to-event variable: events; rate

4.2 Main exposure sensitivity to adjustment variables

The function `coxreg_change` calculates the hazard ratio for the main (binary) exposure under permutations of the full adjustment model. The return value is a data frame grouped by the column `change` whose values indentidy

- `univariate` effect (on main exposure) of adjusting for 1 variable (in isolation)
- `full` effect of subtracting 1 variable (in isolation) from the full model
- `sequential_inclusion` effect of adding variables sequentially (from unadjusted to the full model)
- `sequential_exclusion` effect of subtracting variables sequentially from the full model

```
coxreg_change(
    data = OD,
    surv = data.frame(label="U",time="t.U",event="ev.U"),
                              ## 'stab' or common name of surv-pairs
    main = "E",               ## exposure term
    terms = c("male", "age"), ## adjustment variables
    uni = TRUE,               ## univariate effects?
    full = TRUE,              ## fully adjusted effects?
    inc = TRUE,               ## sequential inclusion?
    exc = TRUE                ## sequential exclusion?
)
```

```
##       term       HR      HR.l     HR.h               change
##     <char>    <num>     <num>    <num>               <char>
##  1: (none) 1.010557 0.8045196 1.269362          univariate
##  2:   male 1.041653 0.8287443 1.309259          univariate
##  3:    age 1.159075 0.9144578 1.469128          univariate
##  4: (none) 1.210030 0.9527828 1.536733                full
##  5:   male 1.159075 0.9144578 1.469128                full
##  6:    age 1.041653 0.8287443 1.309259                full
##  7: (none) 1.010557 0.8045196 1.269362 sequential_inclusion
##  8:    age 1.159075 0.9144578 1.469128 sequential_inclusion
##  9:   male 1.210030 0.9527828 1.536733 sequential_inclusion
## 10: (none) 1.210030 0.9527828 1.536733 sequential_exclusion
## 11:    age 1.041653 0.8287443 1.309259 sequential_exclusion
## 12:   male 1.010557 0.8045196 1.269362 sequential_exclusion
```

4.3 Confounder bias

In observational studies the major difficulty is the possibility of (unknown) counfounding. The function `coxreg_bias` examines the effect of the main exposure - assumed to be binary - if additional coundfounding variables could have been adjusted for.

```
coxreg_bias(
    data = OD,
    surv = "U",
    main = "E",     ## exposure term
    bnry = "male", ## binary adjustment variables
    real = "age"   ## real (continuous) adjustment variables
)[, c(1:2, 4:6, 9:11)]

## Key: <term>
##      term   type      stat0      stat1    adjHR   mainHR  mainHR.l mainHR.u
##    <char> <char>      <num>      <num>    <num>    <num>     <num>    <num>
## 1:      E   main  0.0000000  1.0000000 1.210030       NA        NA       NA
## 2:    age   real 62.0128175 55.6471297 1.022791 1.396679 1.0997555 1.773769
## 3:   male   bnry  0.5336538  0.4547872 1.509246 1.249489 0.9838574 1.586839
```

In the output we see the mean value of adjustment variables in the two binary states of the exposure (`stat0` and `stat1`, respectively). We can see what the coefficients are in an adjusted analysis for each term (`adjHR`), but more interestingly we can see what the coefficient on the main exposure would be (`mainHR`) if we could adjust for an additional confounder with the same association to both the outcome and exposure as the given variables, as well as the lower and upper part of the confidence interval (`mainHR.l` and `mainHR.u`, respectively).

We can "manually" add hypothetical confounder with a given association to the outcome - given as a hazard ratio - and to the exposure - given as the mean level at the 2 different states.

```
cb <- coxreg_bias(data = OD,
                  surv = "U",
                  main = "E",
                  bnry = "male",
                  real = "age",
                  bnry.manual = list("confB" = c(1.3, .55, .45)),
                  real.manual = list("confR" = c(1.01, 50,  50)))
cb[, c(1, 3:5, 9:11)]

## Key: <term>
##      term manual      stat0      stat1   mainHR  mainHR.l mainHR.u
##    <char>  <num>      <num>      <num>    <num>     <num>    <num>
## 1:      E      0  0.0000000  1.0000000       NA        NA       NA
## 2:    age      0 62.0128175 55.6471297 1.396679 1.0997555 1.773769
## 3:  confB      1  0.5500000  0.4500000 1.242013 0.9779708 1.577345
## 4:  confR      1 50.0000000 50.0000000 1.210030 0.9527870 1.536727
## 5:   male      0  0.5336538  0.4547872 1.249489 0.9838574 1.586839

r <- attr(cb, "tidy")
```

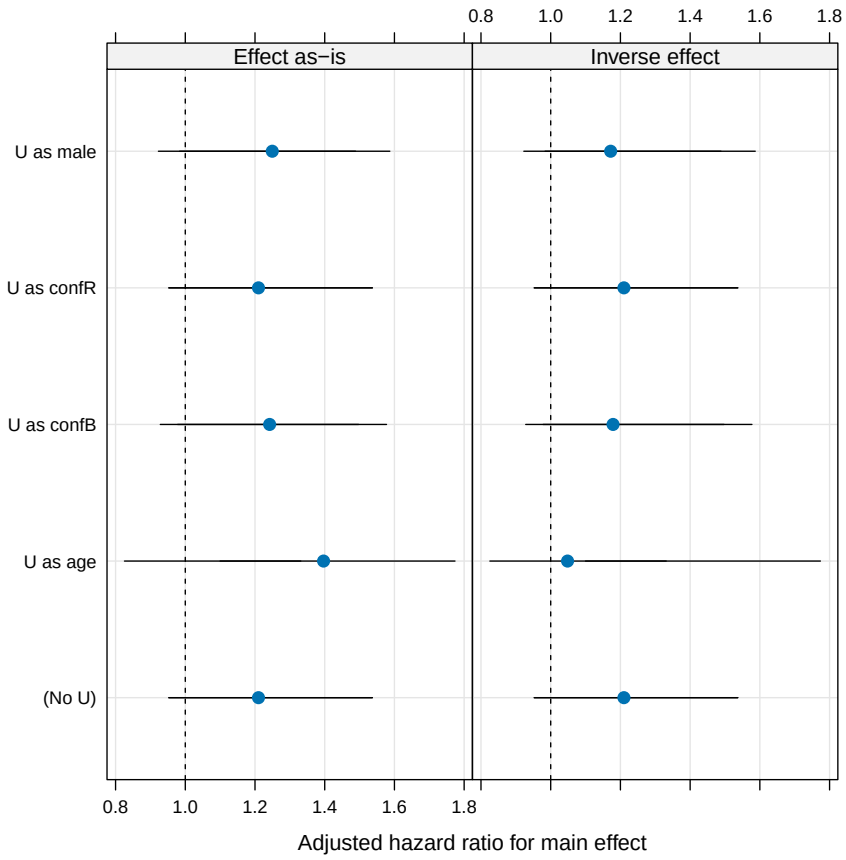

Figure 6: Adjusted hazard ratio for main effect when adjusting for age and sex, as well as a confounder U specified in the figure.

4.4 Miscellanous helpers mostly for time dependent covariates

Maybe document these functions here at some point:

- `event1trunc`
- `tstart2zero`
- `time_num2date` and `time_date2num`
- `cbin`
- `state2event`
- `tdc_statistic`, `tdc_calculator`, and `tdc_count`